Foundations of Machine Learning

Assignment 3: Mini-ML Library and Autograd Engine

P V Charan Teja
AI24BTECH11022

Contents

1 Problem 1: Library Structure **2**

2 Problem 2: Logistic Regression using SGD **3**

3 Problem 3: Spam Classification Experiment **3**

4 Problem 4: Autograd Engine & Visualization **4**

5 Problem 5: Capstone Showdown Analysis **5**

1 Problem 1: Library Structure

Objective: To create a modular, reusable `my_ml_lib/` package that includes datasets, preprocessing, linear models, model selection utilities, and neural network modules with autograd.

Structure:

```
my_ml_lib/
 __init__.py
 datasets/
    __init__.py
    _loaders.py
    _synthetic.py
 linear_models/
    __init__.py
    classification/
        __init__.py
        _logistic.py
 model_selection/
    __init__.py
    _split.py
    _kfold.py
 preprocessing/
    __init__.py
    _data.py
    _gaussian.py
    _polynomial.py
 nn/
     __init__.py
     autograd.py
     modules/
        __init__.py
        base.py
        linear.py
        activations.py
        containers.py
     optim.py
     losses.py

train_model.py
visualize.py
capstone_showdown.ipynb
README.md
```

Each module is structured to mirror `scikit-learn`'s interface for better reusability.

—

2 Problem 2: Logistic Regression using SGD

Objective: Implement L2-Regularized Logistic Regression using Stochastic Gradient Descent (SGD).

Mathematical Formulation

The regularized loss function is:

$$L(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log h_i + (1-y_i)\log(1-h_i)] + \frac{\alpha}{2N}\|\mathbf{w}\|_2^2$$

where:

$$h_i = \sigma(\mathbf{w}^T\mathbf{x}_i), \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

SGD Update Rule:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\left(\frac{1}{B}X_B^T(h_B - y_B) + \frac{\alpha}{B}\mathbf{w}\right)$$

where η is the learning rate, B is the mini-batch size, and α is the regularization strength.

Implementation Notes

- Supports mini-batch SGD with configurable learning rate and regularization. - Bias term excluded from regularization. - Convergence controlled via number of epochs.

—

3 Problem 3: Spam Classification Experiment

Goal: To compare the performance of SGD-based logistic regression with and without feature standardization.

Methodology

1. Load data using `load_spambase()`.
2. Split 80/20 using `train_test_split()`.
3. Train model using raw features and standardized features.
4. Perform 5-fold cross-validation to find best α.

Results

Preprocessing	Best α	Train Error	Test Error
Raw	0.1	0.4939	0.4859
Standardized	0.01	0.0826	0.0902

Table 1: Comparison of results for raw vs standardized features.

Observation: Standardization improves numerical stability and convergence of SGD by ensuring all features contribute equally to gradient updates.

—

4 Problem 4: Autograd Engine & Visualization

Objective: Implement a minimal autograd engine similar to PyTorch to automatically compute gradients via backpropagation.

Implemented Operations

add, sub, mul, div, pow, matmul, relu, sigmoid, softmax, exp, log, sum, mean

Backward Pass: Each `Value` node stores: - Its data (`.data`) - Gradient (`.grad`) - Operation (`._op`) - References to previous nodes (`._prev`)

Computation Graph Visualization

The following figure shows the graph produced using `visualize.py` for a simple model:

Sequential(Linear(2, 3), ReLU())

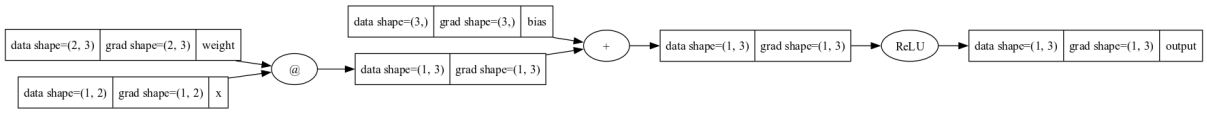

Figure 1: Computation graph visualization of a forward pass through a simple MLP.

Explanation: The nodes represent intermediate `Value` objects, and edges represent operations such as matrix multiplication and activation.

—

5 Problem 5: Capstone Showdown Analysis

Objective: Evaluate multiple model architectures on the Fashion-MNIST dataset to identify the best generalizing model.

Model Types

1. OvR Logistic Regression (SGD)
2. Softmax Regression (raw)
3. Softmax + Polynomial Features
4. Softmax + Gaussian Basis Features
5. MLP (Sequential with ReLU)

Loss Curves

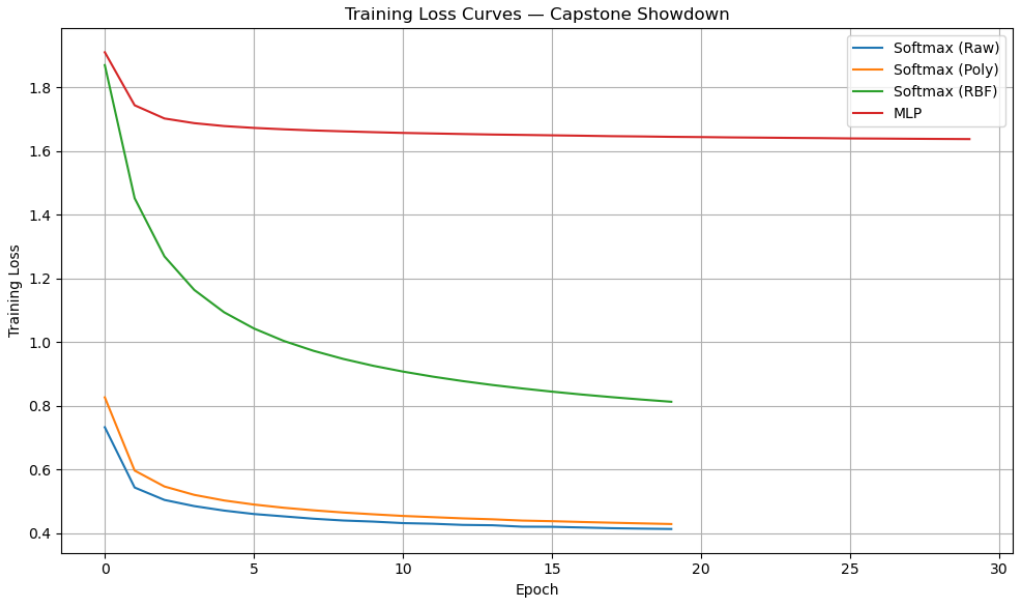

Figure 2: Training loss vs epochs for the four autograd-based models.

Results Table

Model	Validation Accuracy (%)	Test Accuracy (%)
OvR Logistic	—	75.65
Softmax (Raw)	71.99	70.18
Softmax + Poly	76.78	77.95
Softmax + Gaussian	72.41	72.07
MLP (ReLU)	82.31	82.11

Table 2: Best accuracies obtained for each model configuration.

Discussion

a) **OvR vs Softmax:** Softmax regression performed slightly better as it learns class relationships jointly.

b) **Polynomial & Gaussian Features:** Both improved accuracy by enabling non-linear decision boundaries.

c) **MLP Performance:** The MLP achieved the highest accuracy due to its ability to learn complex non-linear features, consistent with the Universal Approximation Theorem.

d) **Hyperparameter Tuning:** Learning rate and number of epochs were most influential. Over-regularization reduced performance.